

LLM101n-BR

Gabaritos comentados

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 30/07/2026

Gabarito — Capítulo 01

Respostas discursivas. Os **números** vêm de `gabarito.py`, que roda os experimentos; rode-o e compare com os seus resultados.

E1 — Leitura de código

1. Por que `vocab_size` é 27, e não 26? São as 26 letras `a-z` mais o token especial de fronteira `.`, que marca início e fim de nome. Sem ele o modelo não saberia com que letra começar nem quando parar.

2. O que aconteceria trocando `(N + 1)` por `N` no cálculo da *loss*? Bigramas nunca vistos teriam contagem 0, logo probabilidade 0, e `log(0) = -infinito` — a *loss* viraria infinita.

Um detalhe que o `gabarito.py` mostra e que vale entender: com este dataset, avaliando **nos mesmos dados do treino**, a *loss* com `+0` sai finita (2,1473). Isso porque todo bigrama avaliado foi, por definição, visto no treino. O infinito aparece assim que você avalia um nome **nov**o que contenha um par inédito. É por isso que se suaviza mesmo quando "parece funcionar".

3. Para que serve o `manual_seed`? Fixa o gerador de números aleatórios, tornando a execução **reprodutível** — você e eu vemos os mesmos nomes gerados. Útil para estudar e depurar; em produção você deixaria aleatório.

E2 — Mexa nos dados

Medido (`gabarito.py`):

Dataset	Loss
155 nomes originais	2,3832
+ 5 nomes estranhos (<code>zwqk</code> , <code>xyzabc</code> ...)	2,3982 (+0,0150)
+ 5 nomes típicos (<code>ana</code> , <code>maria</code> ...)	2,3750 (-0,0081)

A resposta correta é "depende", e é isso que o exercício quer ensinar. Não existe "mais dados = *loss* menor". O que existe é: dados que **seguem o padrão** que o modelo já capturou reduzem a *loss*; dados com padrões **raros** a aumentam, porque o modelo precisa gastar probabilidade em bigramas incomuns.

Se você acrescentou nomes de amigos (portugueses, típicos), provavelmente viu a *loss* cair um pouco. Se acrescentou nomes inventados esquisitos, viu subir.

E3 — O efeito da suavização

Suavização	Loss
+0	2,1473
+1	2,3832
+10	2,8674
+100	3,2114

1. A loss **cresce** monotonicamente com a suavização. Suavizar afasta o modelo dos dados observados.
2. Os nomes gerados ficam mais **genéricos e aleatórios** conforme a suavização aumenta — a distribuição tende ao uniforme, e o modelo "esquece" o que aprendeu. Com +0 eles grudam mais nos padrões do dataset.
3. Ver E1.2 acima: +0 produz probabilidade zero para pares não vistos, e o $\log(0)$ quebra a conta assim que você sai dos dados de treino.

E4 — Temperatura no sampling

Medido, com a mesma semente:

T	Nomes gerados
0,3	celia, lo, la, ria, ca
1,0	cexzma, ogosuraro, zktahwelo, imjtta
3,0	cexzm, loglkurkicczktyh, mvlzimjttainrlkf

1. T **baixo** -> nomes mais "óbvios", curtos e repetitivos: a temperatura concentra a massa de probabilidade nas opções mais prováveis. T **alto** -> mais aleatórios e impronunciáveis: ela achata a distribuição.
2. É exatamente o parâmetro **temperature** das APIs de LLM. Note a forma usada: $p^{**}(1/T)$. Com $T < 1$ o expoente é maior que 1, o que **acentua** as diferenças entre as probabilidades; com $T > 1$ ele as **achata**. Nos modelos modernos a mesma operação é aplicada aos *logits* antes do softmax (dividir os logits por T), o que é matematicamente equivalente.

E5 — Igualando contagem e rede neural

Configuração	Loss
--------------	------

Contagem +1	2,3832
Contagem +0,01	2,1518
Rede, reg=0.01 , 200 passos	2,1774
Rede, reg=0.0 , 1000 passos	2,1600

1 e 2. Com a suavização **igualada** dos dois lados, os valores convergem para a mesma faixa (~2,15-2,18).

3. É a confirmação da tese do capítulo: contar e otimizar chegam ao **mesmo modelo**. A diferença que se via na apostila (2,38 contra 2,21) vinha só do fato de que o **+1** é uma suavização **muito mais forte** que o **reg=0.01**. São o mesmo botão, em escalas diferentes.

E6 — NLL de um nome específico

Solução em `e6_nll_de_nome.py`. Resultados:

```
ana -> NLL = 1.7871
maria -> NLL = 1.7626
vinicius -> NLL = 2.4613
xwq -> NLL = 3.6696
kffkz -> NLL = 3.4179
```

- 1.** `ana` tem NLL bem menor que `xwq` — o modelo considera `ana` muito mais provável.
- 2.** Nomes com bigramas comuns em português (`ma`, `ri`, `an`) pontuam bem. Nomes com sequências raras (`xw`, `kz`) pontuam mal. `vinicius` fica no meio: tem `vi` e `ni` comuns, mas `ci` seguido de `us` é menos frequente no dataset.

E7 — Rumo ao trigrama

Modelo	Suavização	Loss
Bigrama	+0,01	2,1518
Trigrama	+1,0	2,3449
Trigrama	+0,01	1,1523

- 1.** A tabela passa de $27 \times 27 = 729$ para $27 \times 27 \times 27 = \textbf{**19.683**}$ células.
- 2.** Com suavização comparável (**+0,01**), a loss **melhora muito**: 2,15 -> 1,15. Mais contexto ajuda de verdade.

Atenção ao efeito da suavização aqui: com $+1$, o trigrama fica *pior* que o bigrama (2,34 contra 2,15). Isso não é o trigrama sendo ruim — é que somar 1 a **19.683** células injeta muito mais "massa uniforme" do que somar 1 a 729. Quanto maior a tabela, mais destrutiva é a mesma suavização. Comparar modelos com suavização fixa é comparar coisas diferentes.

3. O problema novo é a **esparsidade**: a maioria das células nunca é preenchida. A suavização evita o $\log(0)$, mas **não inventa informação** — para combinações nunca vistas o modelo apenas chuta uniformemente. E isso piora exponencialmente: com k caracteres de contexto são $27^{(k+1)}$ células.

É exatamente essa parede que motiva o **Capítulo 3**: em vez de contar células, o MLP aprende uma representação **densa e compartilhada**, onde contextos parecidos se apoiam mutuamente.

Gabarito — Capítulo 02

Respostas discursivas. Os **números** vêm de `gabarito.py`.

E1 — Leitura de código

1. Para `f = a * b` com `a = 3`, `b = 4`: `df/da = b = 4` e `df/db = a = 3`. A derivada em relação a um fator é o **outro** fator.
2. Para `d = a*b + c`: `dd/dc = 1`. A soma não altera o gradiente porque a derivada de uma soma em relação a cada parcela é 1 — ela apenas **distribui** o gradiente que chega, sem escalá-lo. É por isso que, no código, o `_backward` da soma é `self.grad += 1.0 * out.grad`.
3. Por que **tanh saturada tem gradiente quase zero**: a derivada é `1 - tanh(x)2`. Para `x` grande, `tanh(x) -> ±1`, logo `tanh(x)2 -> 1` e a derivada `-> 0`. Nas regiões extremas a curva é praticamente horizontal — mexer na entrada quase não muda a saída, e o gradiente morre. É o mesmo efeito que reaparece no softmax do Capítulo 4 (daí o fator `1/√d`) e na inicialização do Capítulo 7.

E2 — Conferindo com diferença finita

Variável	Autograd	Diferença finita	Erro relativo
<code>a</code>	-0,00402285	-0,00402285	5,97e-10
<code>b</code>	+0,00268190	+0,00268190	5,97e-10
<code>c</code>	+0,00134095	+0,00134095	5,97e-10

Batem. Dois detalhes que valem a pena:

- Usamos a **diferença central**, `(f(x+h) - f(x-h)) / 2h`, e não a ingênua `(f(x+h) - f(x)) / h`. A central tem erro `O(h2)` em vez de `O(h)` — bem mais precisa pelo mesmo custo.
- Existe um **h ótimo**: grande demais e a aproximação da derivada é ruim; pequeno demais e o cancelamento em ponto flutuante domina (você subtrai dois números quase iguais). Por volta de `1e-6` costuma ser um bom meio-termo em `float64`.

Esta é a forma clássica de **testar** um autograd, e é o que bibliotecas de verdade usam nos seus testes (`torch.autograd.gradcheck` faz exatamente isso).

E3 — Implementando `log()`

1. A derivada local é `d(ln x)/dx = 1/x`.

2. Seguindo o padrão de sempre — derivada local vezes o gradiente que vem de fora:

```
def log(self):
    out = Value(math.log(self.data), (self,), "log")
    def _backward():
        self.grad += (1.0 / self.data) * out.grad
    out._backward = _backward
    return out
```

3. Verificado: $\log(3.0) = 1,098612$ (bate com `math.log`), e $d(\log)/dx = 0,333333$ contra `0,333333` da diferença finita — erro de $4,66e-11$.

Cuidado que o exercício não pede mas vale saber: `log` só é definido para $x > 0$. Numa implementação séria você trataria $x \leq 0$ explicitamente, em vez de deixar o `math.log` levantar exceção no meio de um treino.

E4 — Por que zerar o gradiente

Medido, 20 passos:

passo 0	passo 5	passo 19
<code>cost</code> 0,929	0,0006	0,0001
<code>zero_grad</code>		
<code>sen</code> 0,929	0,7659	0,6158
<code>zero_grad</code>		

1 e 2. Leia os números, não a intuição. Sem o `zero_grad` o treino **não explode neste caso** — ele **estagna**. A loss cai até ~0,6 e trava, enquanto com `zero_grad` chega a 0,0001.

Por que estagna em vez de divergir? Os gradientes são acumulados com `+=`, então o gradiente do passo N é a soma de todos os anteriores. Isso produz dois efeitos que se combatem: o passo fica **grande demais** (empurra para divergir) mas também **desatualizado** — carrega direções calculadas para pesos que já mudaram. Aqui o modelo entra num vai-e-vem e não converge. Com learning rate maior, ou em outros problemas, o mesmo bug realmente diverge.

3. É exatamente o `w.grad = None` do Capítulo 1, pelo mesmo motivo.

A lição que mais importa: o resultado fica errado de um jeito que **não se parece com um erro**. Não há exceção, não há `NaN` — só um treino pior. Bugs assim são os mais caros de encontrar, e é por isso que se compara sempre contra uma referência.

E5 — Bate com o PyTorch?

Solução em `e5_check_vs_torch.py`. Resultado: `ALL MATCH: True`, com diferença máxima de **8,94e-08**.

2. Trocando `tanh` por `relu`, continuam batendo — o mecanismo é o mesmo, só muda a derivada local registrada por aquela operação.

E6 — Arquitetura e learning rate

Arquitetura	Parâmetros	Loss final	Situação
[4, 4, 1]	41	0,000000	convergiu
[8, 1]	41	0,000000	convergiu
[16, 16, 1]	353	1,38e+147	DIVERGIU

Learning rate	Loss final	Situação
0,5	1,80e+248	DIVERGIU
0,05	0,000000	boa
0,001	0,136863	lenta demais

1. A primeira resposta contraria o senso comum. [4,4,1] e [8,1] convergem — e, por coincidência, têm exatamente os **mesmos 41 parâmetros**. Mas [16,16,1], com 353 parâmetros, **diverge** com a mesma learning rate.

Rede maior não é automaticamente melhor. Com mais parâmetros, a soma dos gradientes fica maior, e uma `lr` que servia para a rede pequena passa a ser grande demais. **A learning rate adequada depende do tamanho do modelo** — que é justamente o que motiva a inicialização escalada e o agendamento do Capítulo 7.

2. A curva em U clássica: `0,5` diverge, `0,05` converge, `0,001` quase não anda no orçamento de 100 passos. O Capítulo 7 mede isso de forma sistemática (exercício E5 de lá).

3. Trocar `tanh` por `relu` neste problema faz pouca diferença: com 4 pontos de dados, qualquer não-linearidade razoável resolve. A diferença entre ativações aparece em redes profundas, e é o assunto da Seção 2 do Capítulo 7.

E7 — Visualizando o grafo

Implementação em `gabarito.py`. Para `(a*b + c).tanh()`:

```
Value(data=0.9993, grad=1.0000) [tanh]
+-- Value(data=4.0000, grad=0.0013) [+]
+-- Value(data=10.0000, grad=0.0013)
```



```
+- - Value(data=-6.0000, grad=0.0013) [*]  
+- - Value(data=2.0000, grad=-0.0040)  
+- - Value(data=-3.0000, grad=0.0027)
```

Leia de cima para baixo: a raiz é a saída, e cada nível mostra os "pais" que a produziram. A `backward()` percorre exatamente esta estrutura, aplicando a regra da cadeia.

Repare em como o gradiente se comporta:

- O `tanh` na raiz tem `grad = 1.0` (é a semente, $d(\text{saída})/d(\text{saída}) = 1$).
- A **soma** distribui `0,0013` igualmente para `c` e para o produto — sem alterar nada.
- A **multiplicação** escala: `a` recebe $0,0013 \times (-3) = -0,0040$ e `b` recebe $0,0013 \times 2 = 0,0027$.

Cada um multiplicado pelo valor do outro.

E note que todos os gradientes são pequenos ($\sim 1e-3$): a `tanh` está saturada em `x = 4`, o que confirma o que a resposta do E1.3 previu.

Gabarito — Capítulo 03

Respostas discursivas. Os **números** vêm de `gabarito.py`.

Sobre o orçamento: o gabarito treina **4.000 passos** (a apostila usa 20.000) para rodar em poucos minutos. Os valores absolutos ficam piores que os da apostila, mas as **comparações entre configurações** — que é o que os exercícios pedem — continuam válidas. Onde essa diferença muda a conclusão, está dito explicitamente (veja o E4).

E1 — Leitura de código

1. Por que `emb = C[X]` tem formato `(N, 3, 10)`? `X` tem formato `(N, 3)`: `N` exemplos, 3 índices de caractere cada. `C[X]` substitui cada índice pela linha correspondente de `C`, que tem 10 números. Então: `N` exemplos $\times$ 3 posições de contexto $\times$ 10 dimensões de embedding.
2. Por que o `.view(N, -1)` antes da camada linear? A camada linear espera um vetor por exemplo, não uma matriz `3 $\times$ 10`. O `view` reorganiza os mesmos números em `(N, 30)`, colando os três embeddings num vetor só. O `-1` diz ao PyTorch "calcule esta dimensão". Note que `view` não copia dados — apenas reinterpreta o mesmo bloco de memória.
3. O que `F.cross_entropy(logits, Y)` faz? Exatamente o que fizemos à mão no Capítulo 1: **softmax** seguido de **negative log-likelihood**. A diferença é que a versão do PyTorch é numericamente estável (evita `exp()` de números grandes, que estouraria) e mais rápida.

E2 — O efeito do contexto

<code>block_size</code>	Parâmetros	Treino	Validação
1	7.897	2,3498	2,3401
3	11.897	2,0508	2,0452
5	15.897	2,0165	2,0114

1. Mais contexto **ajuda**: a validação cai de 2,34 para 2,01.
2. Com `block_size = 1` o MLP vira um **bigrama neural** — ele só vê o caractere anterior, como no Capítulo 1. E de fato a loss (2,34) cai na faixa do bigrama daquele capítulo (~2,4). É uma boa confirmação de que os dois modelos são a mesma coisa quando o contexto é o mesmo.
3. Sim, os parâmetros crescem: a primeira camada tem `(n_embd  $\times$  block_size)  $\times$  n_hidden` pesos. Cada caractere a mais de contexto acrescenta `n_embd  $\times$  n_hidden = 2.000` pesos.

É exatamente esse custo linear que motiva a **atenção** do Capítulo 4: lá, aumentar o contexto **não** aumenta o número de parâmetros.

E3 — Tamanho da camada oculta

<code>n_hidden</code>	Parâmetros	Treino	Validação
50	3.197	2,1128	2,1023
200	11.897	2,0508	2,0452
500	29.297	2,0287	2,0271

1. Mais neurônios, mais parâmetros, loss menor.

2. **Retorno decrescente, e claro:** de 50 para 200 (parâmetros $\times 3,7$) a validação melhora 0,057. De 200 para 500 (parâmetros $\times 2,5$) melhora só 0,018 — três vezes menos ganho. Cada parâmetro novo contribui menos que o anterior.

E4 — Learning rate

<code>lr</code>	Validação	Situação
0,001	2,4547	lenta demais
0,01	2,2162	
0,1	2,0452	<- o valor da apostila
1,0	2,0019	melhor aqui
10,0	—	divergiu
50,0	—	divergiu

1. Esta resposta provavelmente contraria o que você esperava. A curva é em U, mas o mínimo **não** está no **0,1** que a apostila usa — neste orçamento de 4.000 passos, `lr = 1,0` vai melhor. A divergência só aparece em **10,0**.

Isso **não** significa que a apostila esteja errada. Ela treina 20.000 passos, e com orçamento maior uma learning rate menor tem tempo de refinar e chega mais longe. A conclusão correta:

A melhor learning rate depende do orçamento de passos. Ela não é uma propriedade do modelo. O mesmo fenômeno reaparece no Capítulo 7 (exercício E5), onde $3e-3$ ganha de $1e-3$ justamente porque o experimento treina menos passos que a apostila.

2. Removendo o decaimento com a melhor **lr**: 2,0915 sem contra **2,0019** com — o decaimento vale **0,09** de loss. Passos menores no fim permitem "assentar" no mínimo em vez de ficar pulando em torno dele.

E5 — Overfitting com dataset pequeno

Solução em `e5_overfitting.py`:

```
=== DATASET PEQUENO (155 nomes) ===  
loss treino = 0.8001  
loss validacao = 6.5086
```

1. Um abismo: treino 0,80 contra validação 6,51.
2. O mesmo modelo generaliza com 64 mil nomes e decora com 155 porque o número de **parâmetros** (11.897) é maior que o número de **exemplos de treino** (880). Com essa proporção, memorizar é mais fácil que aprender o padrão — e o modelo faz o que é mais fácil. (É a mesma conta que reaparece no Capítulo 11: 0,28 token por parâmetro.)
3. Os nomes gerados se parecerem demais com nomes reais do dataset é **mau** sinal porque significa que o modelo está **reproduzindo**, não **generalizando**. Um bom modelo gera nomes que *poderiam* estar na lista mas não estão. Qualidade aparente na saída pode esconder memorização — por isso se mede na validação, não no olho.

E6 — Visualizando os embeddings

Com `n_embd = 2` (treino 2,2954 / validação 2,2873 — pior que com 10, como esperado):

Distância média	Valor
vogal <-> vogal	0,732
vogal <-> consoante	1,173
razão	1,60x

1. **As vogais ficam agrupadas.** Elas estão 1,6x mais próximas entre si do que das consoantes. A rede descobriu sozinha que vogais têm papel parecido — **só por prever o próximo caractere**. Ninguém ensinou fonética a ela.

Este é um dos resultados mais bonitos do curso: estrutura linguística emergindo de pura estatística de sequência.

2. Reduzir para `n_embd = 2` piora a loss (menos capacidade por caractere), mas é o único jeito de desenhar os embeddings num plano. É um trade-off comum em interpretabilidade: sacrifica-se desempenho para poder **ver**. (Em modelos de verdade usa-se PCA ou t-SNE para projetar embeddings de centenas de dimensões em 2, sem retreinar.)

E7 — Contando os parâmetros

Componente	C Total
<code>C</code> (embeddings)	27270 × 10
<code>W1</code>	306.000 × 200
<code>b1</code>	20000
<code>W2</code>	206.400 × 27
<code>b2</code>	2727
Total	11.897

1. Fórmula geral:

```
vocab×n_embd + (n_embd×block_size)×n_hidden + n_hidden + n_hidden×vocab + vocab
```

2. Confere exatamente com o **11897** que o código informa.

3. **Quem domina:** `W1` (6.000) e `W2` (5.400) somam **96%** do total. As duas matrizes grandes são o modelo; embeddings e vieses são detalhes.

Consequências: - Dobrar `n_embd` quase dobra o total (ele aparece em `C` e em `W1`). - Dobrar `n_hidden` também (aparece em `W1` e em `W2`). - Dobrar `block_size` só afeta `W1` — cresce menos que os outros dois.

Gabarito — Capítulo 04

Respostas discursivas. Os **números** vêm de `gabarito.py`.

Orçamento: 4.000 passos por configuração (a apostila usa 20.000). Valores absolutos ficam piores; as comparações continuam válidas.

E1 — Leitura de código

1. O que a `tril` impede, e o que aconteceria sem ela no treino? Ela impede que uma posição veja o **futuro**. Sem ela, ao prever o token seguinte a posição poderia consultar justamente a resposta — a loss de treino despencaria e o modelo não aprenderia nada útil, porque na hora de gerar texto o futuro não existe. É *data leakage*.

(O exercício E2 mostra que isso tem uma sutileza importante — veja abaixo.)

2. Por que dividir por $\sqrt{\text{head_size}}$? O produto escalar $q \cdot k$ é a soma de `head_size` produtos, então a sua magnitude cresce com a dimensão. Pontuações grandes fazem o softmax **saturar**: a distribuição vira quase um one-hot e o gradiente morre. Dividir por $\sqrt{\text{head_size}}$ mantém a variância em torno de 1 independentemente do tamanho da cabeça.

3. Query, key e value: - **query**: "o que eu estou procurando?" - **key**: "o que eu tenho a oferecer?" (o endereço pelo qual sou encontrado) - **value**: "o que eu de fato entrego, se você me escolher" (o conteúdo)

Separar *key* de *value* dá liberdade ao modelo: como sou encontrado e o que eu entrego são coisas diferentes.

E2 — Sem a máscara causal

Este exercício tem uma armadilha que vale mais que a pergunta.

Configuração	Treino	Validação
1 camada, máscara ON	2,1589	2,1517
1 camada, máscara OFF	2,1589	2,1517
2 camadas, máscara ON	2,2238	2,2122
2 camadas, máscara OFF	2,1614	2,1513

Com uma camada, remover a máscara não muda absolutamente nada. Os números são idênticos até a quarta casa. Não é erro de medição.

O motivo é estrutural: este modelo usa **apenas a última posição** para prever, e a última linha da matriz triangular **já é toda de uns**. Para a posição que importa, a máscara nunca fez diferença — ela só afetava posições intermediárias, cujas saídas são **descartadas**.

Com duas camadas a história muda. A saída das posições intermediárias da camada 1 alimenta a camada 2, e a última posição da camada 2 as lê. Aí o futuro realmente vaza, e a loss sem máscara fica artificialmente melhor (2,1513 contra 2,2122).

E onde a máscara é absolutamente essencial: quando o modelo prevê em **todas** as posições (Capítulo 11). Ali a posição **t** veria literalmente o token **t+1** que deve prever — vazamento direto e devastador.

A lição: uma proteção pode estar **correta** e ainda assim ser **inócua** no desenho atual. Removê-la e não ver diferença **não prova que é inútil** — prova que o seu teste não exercita o caminho que ela protege. É exatamente assim que proteções de segurança são removidas "porque não faziam nada".

E3 — Sem o embedding posicional

T	Validação
co	12,317
posicional	
se	20,159
posicional	
	+0,0442

1 e 2. A loss piora. A atenção é **invariante a permutações**: ela calcula afinidades entre pares de posições, mas nada no mecanismo diz **qual veio antes**. Sem o embedding posicional, **ana** e **naa** viram o mesmo conjunto de tokens, e o modelo perde a capacidade de usar a **ordem** — que em linguagem é quase tudo.

A penalidade de 0,044 pode parecer modesta, mas note: para *nomes*, boa parte do sinal está em *quais* letras aparecem (estatística de caracteres), não só na ordem. Em prosa, o mesmo experimento seria muito mais destrutivo.

E4 — Sem a escala $1/\sqrt{\text{head_size}}$

Treino	Validação	Peso máximo médio de atenção
--------	-----------	------------------------------

co 1589	2,1517	0,406
escala		
se 3508	2,3428	0,985
escala		

1 e 2. Sem a escala a loss piora bastante (+0,19), e a causa aparece na terceira coluna: o peso máximo por linha sobe de 0,41 para **0,985**. A distribuição de atenção virou praticamente um **one-hot** — o modelo escolhe uma posição e ignora todas as outras.

3. Softmax saturado tem gradiente quase nulo. É exatamente o mesmo efeito da **tanh** saturada do Capítulo 2: o modelo trava numa escolha e perde a capacidade de ajustá-la. A escala não é um detalhe de normalização — é o que mantém o mecanismo **treinável**.

E5 — Tamanho do contexto

block_size	Parâmetros	Treino	Validação
3	11.103	2,2194	2,2110
8	11.363	2,1589	2,1517
16	11.779	2,2122	2,1988

1. A resposta não é "mais contexto é sempre melhor". De 3 para 8 melhora; de 8 para 16 **piora**. Não é retorno decrescente — é retorno **negativo**.

Por quê? Nomes têm ~7 letras. Com **block_size = 16**, a imensa maioria das posições do contexto é **preenchimento** (.), e o modelo gasta capacidade processando padding em vez de sinal.

2. Somando a isso, os embeddings posicionais das posições distantes quase nunca são exercitados: recebem pouco gradiente, permanecem próximos da inicialização aleatória e injetam ruído.

A lição: o contexto deve ser dimensionado pelo **tamanho real do dado**. Contexto grande demais não é neutro — custa computação e qualidade. (No Capítulo 11, com prosa, **block_size = 128** faz sentido justamente porque o texto de verdade é longo.)

3. Dois custos que crescem de formas diferentes: - **computação:** cresce com T^2 (a matriz de afinidades é $T \times T$) - **parâmetros:** cresce só com o embedding posicional — de 11.103 para 11.779, quase nada

É o **oposto** do MLP do Capítulo 3, onde os parâmetros cresciam linearmente com o contexto. A atenção troca custo de parâmetros por custo de computação.

E6 — Inspeccionando os pesos

Solução em `e6_inspecionar_atencao.py`.

O achado principal: boa parte do peso cai nos tokens de **preenchimento** (`.`), não nas letras. Isso **não** é bug — o softmax obriga os pesos a somarem 1, então quando o modelo não precisa trazer informação de nenhuma posição específica, ele "estaciona" o peso numa posição inofensiva. O fenômeno tem nome na literatura de LLMs: **attention sink**.

E7 — Multi-head attention

Cabeças	<code>head_size</code>	Parâmetros	Treino	Validação
1	52	11.363	2,1589	2,1517
2	26	14.119	2,0752	2,0643
4	13	14.119	2,0463	2,0367

1. Várias cabeças pequenas ganham. De 1 para 4 cabeças a validação melhora 0,115 — um ganho grande para este capítulo.

Note que 2 e 4 cabeças têm **exatamente os mesmos 14.119 parâmetros**: `head_size = n_embd // n_head`, então o total de pesos das cabeças é constante. A melhoria de 2 para 4 vem **só da organização**, não de capacidade extra.

A versão de 1 cabeça tem menos parâmetros (11.363) porque não tem a projeção de saída. A comparação limpa é entre 2 e 4 cabeças.

2. Por que várias ajudam? Uma cabeça aprende **um** critério de "onde olhar". Várias aprendem vários — uma pode seguir a vogal anterior, outra o começo da palavra, outra a última consoante. Como as saídas são concatenadas e projetadas, o modelo combina esses critérios.

3. É exatamente a peça que abre o Capítulo 5.

Gabarito — Capítulo 05

Respostas discursivas. Os **números** vêm de `gabarito.py`.

Orçamento: 2.500 passos por configuração (a apostila usa 15.000).

E1 — Leitura de código

1. Por que `x = x + self.sa(self.ln1(x))` e não `x = self.sa(self.ln1(x))`? A soma é a **conexão residual**. Ela muda duas coisas:

- **na ida:** a sub-camada não precisa produzir a representação inteira — só uma **correção** ao que já estava lá. Aprender um ajuste é mais fácil que reconstruir tudo.
- **na volta:** a derivada de uma soma **distribui** o gradiente igualmente. O `x +` cria um caminho pelo qual o gradiente flui até as primeiras camadas **sem ser multiplicado por nada**.

2. Por que `head_size = n_embd // n_head`? Para que a concatenação das cabeças volte a ter `n_embd`. Com `n_embd = 64` e `n_head = 4`, são 4 cabeças de 16 que concatenam de volta em 64 — **quatro cabeças pequenas custam o mesmo que uma grande**. Se cada cabeça tivesse `head_size = n_embd`, o custo seria `n_head` vezes maior.

3. **Atenção vs feedforward:** A **atenção comunica** — move informação entre posições. O **feedforward computa** — processa cada posição isoladamente, sem trocar nada. Um bloco faz as duas coisas: as posições conversam, depois cada uma pensa sozinha sobre o que ouviu.

E2 — Sem as conexões residuais

Solução em `e2_ablacoes.py`:

Configuração	Loss validação
3 blocos, residual ON	1,9029
3 blocos, residual OFF	2,1971 (+0,294)
6 blocos, residual ON	1,8865
6 blocos, residual OFF	2,7728 (+0,886)

A penalidade **triplica** ao dobrar a profundidade. E o mais revelador: **sem** residuais, ir de 3 para 6 blocos **piora** o modelo (2,197 -> 2,773); **com** residuais, melhora (1,903 -> 1,887).

Sem conexões residuais, profundidade atrapalha. Com elas, ajuda.

E3 — Sem LayerNorm

Blocos	lr	LayerNorm	Validação
3	1e-3	ON	1,9365
3	1e-3	OFF	1,9318
3	3e-3	ON	1,9504
3	3e-3	OFF	2,0104
8	1e-3	ON	1,9228
8	1e-3	OFF	1,9181
8	3e-3	ON	1,9709
8	3e-3	OFF	DIVERGIU

Penalidade de remover a LayerNorm:

$lr = 1e-3$	$lr = 3e-3$
3 -0,0047 blocos	+0,0600
8 -0,0047 blocos	divergiu

1 e 2. A resposta não é "LayerNorm sempre ajuda". Leia a tabela de penalidades:

- Com $lr = 1e-3$, remover a LayerNorm dá **-0,0047 nas duas profundidades** — ela é dispensável, e até marginalmente prejudicial, tanto com 3 quanto com 8 blocos.
- Com $lr = 3e-3$, remover custa +0,06 com 3 blocos e **destrói o treino** com 8.

Não é a profundidade que cria a necessidade — é a learning rate. A profundidade **amplifica a falha**: o mesmo lr que apenas degrada um modelo de 3 blocos aniquila um de 8.

3. Faz sentido pela teoria da Seção 4 da apostila: a LayerNorm impede que o erro de escala se acumule. Com passos pequenos as ativações não saem de escala, e não há o que corrigir. Com passos grandes elas saem — e aí o número de camadas decide se o estrago é recuperável.

A lição, que se repete no curso: uma peça pode ser **correta e necessária em geral**, mas dispensável na configuração específica que você está testando. Isso não a torna inútil — torna o seu teste insuficiente para julgá-la.

E4 — Profundidade e número de cabeças

<code>n_layer</code>	<code>n_head</code>	Parâmetros	Validação
1	4	53.915	1,9818
3	4	153.499	1,9365
6	4	302.875	(ver saída)

<code>n_layer</code>	<code>n_head</code>	Parâmetros	Validação
3	1	153.499	1,9618
3	4	153.499	1,9365
3	8	153.499	1,9227

1 e 2. Mais profundidade ajuda, com retorno decrescente.

3. Mudar `n_head` não muda o número de parâmetros — repare que as três linhas da segunda tabela têm exatamente **153.499**. Como `head_size = n_embd // n_head`, o total de pesos das cabeças é constante. O que muda é **como esses pesos são organizados**: quantos softmaxes independentes existem.

E5 — Uma cabeça grande vs várias pequenas

<code>n_head</code>	<code>head_size</code>	Parâmetros	Validação
1	64	153.499	1,9618
8	8	153.499	1,9227
64	1	153.499	1,9137

1 e 2. Várias cabeças pequenas ganham, e com **exatamente os mesmos parâmetros**. A melhora vem só da organização: cada cabeça se especializa numa relação diferente, e a projeção combina o que todas trouxeram.

3. Esta resposta não saiu como eu previa.

Eu esperava um limite: com `n_head = 64` cada cabeça tem dimensão 1, e um produto escalar de vetores de dimensão 1 é só a multiplicação de dois números — pareceria pobre demais para expressar afinidade.

A medição diz o contrário: 64 cabeças foi o melhor dos três.

Por que o limite não aparece aqui? Porque a tarefa é simples (nomes, contexto 8). Sessenta e quatro padrões de atenção escalares, combinados pela projeção, dão conta. O custo de cada cabeça ser pobre é compensado pela quantidade.

Na prática os modelos usam `head_size` entre 32 e 128 — mas por razões que este experimento **não exercita**: tarefas complexas onde cada cabeça precisa comparar representações ricas, e eficiência de hardware (cabeças muito pequenas geram matmuls pequenas, que a GPU odeia — ver Capítulo 8).

Vale saber a regra prática E saber por que ela existe. Uma regra da literatura pode não se reproduzir na sua escala.

E6 — Dropout

Dropout	Treino	Validação	Gap
0,0	1,9438	1,9365	-0,0073
0,1	1,9517	1,9448	-0,0069
0,3	1,9774	1,9681	-0,0093

1. O dropout atrapalha aqui, e de forma monotônica. Com 64 mil nomes, treino e validação já andam juntos — o gap é praticamente nulo. Sem overfitting para combater, o dropout só remove capacidade.

É exatamente a mesma lição do weight decay no Capítulo 7, e o padrão é o mesmo: **treino e validação pioram juntos, e o gap não muda**. Regularização que não está resolvendo nada.

2. Ele ajudaria com **poucos dados** — releia o E5 do Capítulo 3 (155 nomes: treino 0,80 contra validação 6,51).

3. Por que desligar na avaliação? Porque com dropout ativo você estaria medindo um modelo **aleatoriamente mutilado** — a loss viria pior e ruidosa, sem significar nada. O `model.eval()` desliga o dropout (e ajusta a BatchNorm, quando existe).

E7 — Contando os parâmetros

Com `n_embd = 64`, `n_head = 4`, `n_layer = 3`, `block_size = 8`, `vocab = 27`:

Componente	Total
<code>token_emb</code> (27×64)	1.728
<code>pos_emb</code> (8×64)	512

por bloco (qkv + proj + ff + 2 LayerNorm)	49.792
× 3 blocos	149.376
LayerNorm final	128
lm_head (64 × 27 + 27)	1.755
Total	153.499

1 e 2. A fórmula reproduz exatamente o número que o código informa.

3. Quem domina: os blocos somam 149.376 de 153.499 — **97%**. E dentro de cada bloco, o **feedforward** é o maior pedaço, por causa da expansão 4×.

Consequências para escalar: - Dobrar **n_embd** **quadruplica** o custo dos blocos (todas as matrizes são **ne × ne** ou **ne × 4ne**). - Dobrar **n_layer** apenas **dobra**.

É por isso que modelos grandes crescem mais em profundidade do que em largura — e por que **n_embd** é a decisão de arquitetura mais cara.

Gabarito — Capítulo 06

Respostas discursivas. Os **números** vêm de `gabarito.py` (roda em ~2 minutos, sem treinar rede nenhuma).

E1 — Unicode e UTF-8

1. Quantos bytes ocupa "pão"?

```
'pao' : 3 caracteres, 3 bytes -> [112, 97, 111]
'pão' : 3 caracteres, 4 bytes -> [112, 195, 163, 111]
```

Mesmo número de **caracteres**, um byte a mais. O **ã** ocupa **2 bytes** em UTF-8 (**195, 163**), enquanto **a** ocupa 1.

2. `ord("ç")` vale 231 — isso significa que cabe em 1 byte?

Não, e este é o erro clássico do capítulo. 231 é o **code point** (a identidade Unicode do caractere), não a codificação. Em UTF-8, todo code point acima de 127 usa 2 bytes ou mais — **ç** vira **[195, 167]**.

Confundir code point com byte leva a bugs sutis: você conta 3 caracteres e aloca 3 bytes.

3. Por que começar pelos bytes elimina o "desconhecido"?

Existem exatamente **256** valores de byte, e todos estão no vocabulário base. Qualquer texto — de qualquer idioma, mais emoji, mais código-fonte — é uma sequência de bytes, logo é sempre representável. Não há como aparecer algo "fora do vocabulário".

E2 — Tamanho do vocabulário

Vocabulário	Fusões	Tokens (treino)	Compressão	Ganho
300	44	95.043	1,578x	—
512	256	66.936	2,241x	+0,663
1024	768	52.965	2,832x	+0,591

1 e 2. A compressão melhora, com **retorno decrescente**: as primeiras 256 fusões valem +0,663 de compressão; as 512 seguintes valem só +0,591. As fusões mais frequentes são aprendidas primeiro, e as seguintes cobrem casos cada vez mais raros.

3. O custo de um vocabulário maior não está no tokenizador — está no modelo. Duas matrizes crescem com `vocab_size`:

- a tabela de embeddings: `vocab_size × n_embd`

- a camada de saída (`lm_head`): `n_embd × vocab_size`

Dobrar o vocabulário dobra as duas. É um trade-off direto: **sequências mais curtas (bom) contra modelo maior (custo)**. Modelos reais usam entre 32 mil e 200 mil tokens, e essa escolha é uma decisão de arquitetura, não um detalhe.

E3 — As primeiras fusões

```
256: 'an' 261: 'il'
257: 'a\n' 262: 'on\n'
258: 'e\n' 263: 'ar'
259: 'el' 264: 'o\n'
260: 'on' 265: 'er'
```

- Por que ('a', 'n') é a primeira?** Porque `an` é a sequência de duas letras mais frequente em nomes brasileiros: `ana`, `alessandra`, `fernanda`, `luana`, `antonio`...
- Por que tantas fusões terminam em `\n`?** Quatro das dez primeiras contêm quebra de linha. O motivo é o **formato do arquivo**: um nome por linha. A quebra de linha faz parte do padrão de **terminação** dos nomes, e o tokenizador está literalmente aprendendo "como um nome acaba".

Isso revela algo importante: o tokenizador aprende o **formato do arquivo**, não só a língua. Se você mudasse para nomes separados por vírgula, ele aprenderia `,` no lugar. Formatação é dado.

3. Os tokens mais longos:

```
'ilson\n', 'erson\n', 'isson\n', 'ilene\n', 'ilton\n', 'ilane\n',
'laine\n', 'iane\n', 'ison\n', 'iele\n', 'iana\n', 'aldo\n', ...
```

São **sufixos produtivos de nomes brasileiros**. O algoritmo não tem nenhuma noção de morfologia — ele só conta pares. Estatística pura virando estrutura linguística.

E4 — A ordem das fusões importa

Aplicando as fusões numa ordem arbitrária em vez da ordem de aprendizado:

Texto	Ordem correta	Ordem errada	Round-trip
<code>maria eduarda</code>	7	8	[OK]
<code>joao vinicius</code>	9	9	[OK]
<code>ana beatriz de souza</code>	15	15	[OK]
20.000 caracteres	9.137	10.041	+9,89%

1. O **round-trip continua valendo**. O **decode** desfaz qualquer sequência de fusões válidas, então o texto volta igual. A ordem errada **não quebra** o tokenizador — só o deixa pior.

2. A **compressão piora ~10%** em texto de tamanho razoável. Significativo, ainda que não catastrófico.

Repare no contraste entre as duas partes da tabela — e é a lição metodológica do exercício. Nos três textos curtos, dois deram **exatamente o mesmo** número de tokens. Quem medisse só neles concluiria que "a ordem quase não importa". Só com volume o efeito de ~10% aparece de forma confiável. **Amostra pequena esconde efeito real**.

3. **Por que a regra é obrigatória mesmo assim?** Dois motivos:

- Uma fusão tardia pode usar um token criado por uma anterior; aplicá-la antes desmonta a cadeia.
- Sem ordem fixa, o mesmo texto poderia produzir tokens diferentes em execuções diferentes. Um tokenizador precisa ser **determinístico** — senão o modelo recebe entradas inconsistentes.

E5 — O imposto do português

Solução em `e5_bpe_portugues.py`. Resumo: um BPE do **mesmo tamanho** treinado em português com acentos aprende `'ação '`, `'ção '`, `'não '`, `'são '` como tokens únicos, e gasta **53% menos tokens** (64 contra 135 em três frases).

E6 — Tokens fora do domínio

Texto	Bytes	Tokens	Compressão	Round-trip
nomes (mesmo domínio)	2.000	900	2,22x	[OK]
frase em português	60	50	1,20x	[OK]
japonês + emoji	17	17	1,00x	[OK]
código Python	33	29	1,14x	[OK]

1. Fora do domínio a compressão desaba para perto de 1,00x: nenhuma das fusões aprendidas se aplica, então cada byte vira um token.

2. Mas o **round-trip continua True em todos os casos** — e é essa a garantia que importa. O tokenizador **não comprime** o que não conhece, mas **nunca falha**. Japonês, emoji e código-fonte são todos representáveis, porque o vocabulário base são os 256 bytes.

3. Um **tokenizador de palavras** receberia uma palavra japonesa e não teria entrada nenhuma para ela — precisaria emitir `<UNK>`, **perdendo a informação de forma irreversível**. É a diferença entre "comprime mal" e "não representa".

E7 — Integrando com o modelo

Este exercício é mais de raciocínio que de medição.

1. O que muda em `vocab_size` e nos parâmetros? De 27 para 512 (ou 1024). As duas matrizes ligadas ao vocabulário crescem na mesma proporção: embeddings (`vocab_size × n_embd`) e `lm_head` (`n_embd × vocab_size`). Com o modelo do Capítulo 5 (`n_embd = 64`), isso vai de ~3,5 mil para ~65 mil parâmetros só nessas duas.

2. Quanto texto cabe no mesmo `block_size`? Com compressão de 2,24x, uma janela de 8 tokens passa a cobrir ~18 caracteres em vez de 8. **O contexto efetivo mais que dobra sem custo nenhum** — é o principal ganho prático da tokenização.

3. A loss é comparável? Não, e entender por quê é o ponto do exercício.

A cross-entropy mede a incerteza **por token**. Prever entre 27 opções e prever entre 512 são tarefas de dificuldade diferente: o piso de um chute uniforme passa de $\ln(27) = 3,30$ para $\ln(512) = 6,24$. Uma loss de 2,0 significa coisas muito diferentes nos dois casos.

Para comparar de forma justa, converte-se para **bits por caractere** (BPC):

```
BPC = loss_por_token / ln(2) / caracteres_por_token
```

Essa métrica é independente do tokenizador — e é por isso que é usada na literatura para comparar modelos com vocabulários diferentes. É exatamente a armadilha que o Capítulo 11 aponta ao aposentar o benchmark de 1,776.

Gabarito — Capítulo 07

Respostas discursivas. Os **números** vêm de `gabarito.py`.

Orçamento: 2.500 passos por configuração (a apostila usa 15.000). Onde essa redução muda a conclusão, está dito.

Aviso geral deste gabarito: quatro das oito respostas contrariam a expectativa — inclusive a minha, quando escrevi os exercícios. Todas foram corrigidas pela medição.

E1 — Leitura de código

1. O que a correção de viés faz? m e v começam em **zero**. No primeiro passo, $m = (1 - \beta_1) \cdot g = 0,1 \cdot g$ — dez vezes menor que o gradiente real. As médias móveis **subestimam** no início. Dividir por $(1 - \beta_1^t)$ compensa exatamente isso: no passo 1 o divisor é $0,1$, multiplicando m por 10.

Ela importa mais no início porque é lá que o viés é grande. Conforme t cresce, $\beta_1^t \rightarrow 0$, o divisor tende a 1 e a correção **desaparece sozinha**.

2. Weight decay somado ao gradiente vs aplicado ao parâmetro: No Adam original, o termo de decay entrava no gradiente e depois passava pela **divisão por $\sqrt{v}$** . Isso distorce a regularização: parâmetros com gradiente grande recebiam *menos* decay, sem nenhuma razão. O AdamW aplica o decay direto no parâmetro, fora do caminho adaptativo — daí o "W" de *decoupled*.

3. Por que $\sqrt{\text{fan_in}}$ e não fan_in ? Um neurônio soma fan_in produtos independentes. A **variância** de uma soma de n termos independentes cresce com n , logo o **desvio padrão** cresce com $\sqrt{n}$. Como queremos controlar o desvio padrão, dividimos por $\sqrt{\text{fan_in}}$.

E2 — O warmup é necessário?

Warmup	Validação	Norma média (100 primeiros passos)
0	1,9198	1,223
50	1,9211	1,351
200	1,9219	1,498
500	1,9226	1,645

1. Neste modelo o warmup não ajuda — e piora de forma monotônica. Gastar 500 dos 2.500 passos com learning rate reduzida é desperdiçar 20% do orçamento num modelo que não tem problema de estabilidade.

2. Cuidado com a métrica — e eu escolhi a errada.

A norma dos gradientes nos primeiros passos **umenta** com o warmup (1,223 -> 1,645). Seria tentador ler isso como "o warmup deixa o treino mais instável". Não é.

Com warmup a learning rate começa minúscula, então o modelo demora mais para sair da região de inicialização — onde a loss é alta e os gradientes são **naturalmente maiores**. A norma alta mede **lentidão**, não instabilidade. A métrica não testava o que eu queria testar.

3. Então por que o warmup existe? Porque nos primeiros passos **m** e **v** do Adam quase não têm informação. Em modelos **grandes**, com batches grandes, um passo ruim no início pode levar a loss a um regime do qual ela não se recupera.

O warmup é um seguro contra um risco que cresce com a escala. Nesta escala, ele só custa. Isso não o torna errado — torna este modelo insuficiente para julgá-lo.

E3 — Calibrando o gradient clipping

Norma do gradiente sem clipping: **média 1,324, máxima 2,427**.

Clip	Validação	% de passos cortados
0,1	1,9206	100,0%
1,0	1,9205	99,6%
3,0	1,9219	0,0%
100,0	1,9219	0,0%

1. Com **clip = 3,0** (acima da norma típica de 1,32) **nada** é cortado — o comportamento correto de uma rede de segurança que não é acionada.

2. Com **clip = 0,1**, **100% dos passos são cortados**: isso deixa de ser clipping e vira **normalização** do gradiente — o erro documentado na Seção 5 da apostila.

Mas seja honesto com o número: **aqui isso não prejudicou** (1,9206 contra 1,9219 sem clipping — até marginalmente melhor). Normalizar todo gradiente é um algoritmo **diferente**, não necessariamente pior. Ele só não é o que você pensa que está rodando.

O problema de configurar errado nem sempre é perder desempenho. Às vezes é **não saber qual algoritmo você está usando** — e então não conseguir explicar o resultado.

3. Com **clip = 100** nada é cortado e a loss é idêntica à de não ter clipping. Conclusão honesta: **neste modelo o clipping é dispensável**. Não há picos. Ele existe para o caso patológico.

4. O clipping altera só o **tamanho** do gradiente, não a **direção**: todos os componentes são multiplicados pelo mesmo fator. A informação de "para onde ir" é preservada.

E4 — O ganho certo para a GELU

Desvio padrão das ativações na 8ª camada:

Ganho	tanh	ReLU	GELU
1,000	0,2483	0,0493	0,0068
1,414 ($\sqrt{2}$)	0,5515	0,7879	0,5393
1,500	0,5892	1,2635	1,0508
1,667 (5/3)	0,6472	2,9400	2,7308
2,000	0,7253	12,6212	12,2773

1. O ganho que mantém o desvio ~ 1 para GELU é $\approx 1,49$.

2. Fica **entre** o da ReLU (1,414) e o da tanh (1,667), mais perto da ReLU. Faz sentido: a GELU é uma versão suave da ReLU e deixa passar um pouco menos de sinal, precisando de um ganho ligeiramente maior.

3. O PyTorch não tem entrada para GELU:

```
torch.nn.init.calculate_gain('relu') = 1.4142
torch.nn.init.calculate_gain('gelu') -> ValueError
```

Na prática quase ninguém ajusta esse ganho para GELU — porque modelos modernos usam **LayerNorm**, que re-normaliza a cada bloco e torna o valor exato bem menos crítico (a apostila mede isso na Seção 2).

E5 — A curva da learning rate

Solução em `e5_curva_lr.py`. Curva em U, com mínimo em **3e-3** naquele orçamento — maior que o **1e-3** da apostila, porque **a melhor learning rate depende do orçamento de passos**.

E6 — AdamW vs SGD no modelo de verdade

Otimizador	Lr	Validação
AdamW	1e-3	1,9219
SGD	1e-3	2,6488
SGD	1e-2	2,2738

SGD	1e-1	2,0090
SGD	5e-1	1,9294
SGD + momentum	1e-2	2,0105
SGD + momentum	1e-1	1,9213

1. Com a **mesma** learning rate (1e-3), o SGD fica muito atrás: 2,65 contra 1,92. Um passo bom para o AdamW é minúsculo para o SGD.

2. Mas **ajustando a learning rate, o SGD+momentum EMPATA com o AdamW** (1,9213 contra 1,9219). O SGD puro fica logo atrás (1,9294).

Ou seja: neste modelo o AdamW **não é magicamente superior** — ele é superior **com a learning rate padrão**. A grande vantagem prática do Adam não é chegar mais longe, é chegar lá **sem que você precise caçar a learning rate certa**. Isso vale muito quando cada tentativa custa horas de treino.

3. O modelo tem parâmetros de escalas muito diferentes (embeddings, pesos de atenção, ganhos de LayerNorm, vieses). Uma única learning rate não serve para todos — é o experimento do [optimizers.py](#). O Adam normaliza o passo por parâmetro e escapa desse aperto.

E7 — Por que o weight decay atrapalhou

Weight decay	Treino	Validação	Gap
0,00	1,9243	1,9207	-0,0036
0,01	1,9256	1,9219	-0,0037
0,10	1,9382	1,9334	-0,0048
0,50	2,0015	1,9938	-0,0077

1 e 2. Olhe as três colunas juntas. Aumentar o weight decay piora **treino e validação juntos**, e o **gap entre eles quase não muda**. Esse padrão é a assinatura de que a regularização **não está resolvendo overfitting** — ela está apenas removendo capacidade.

Se houvesse overfitting, o esperado seria: treino piora, validação **melhora**, gap diminui. Não é o que acontece.

3. Com gap praticamente nulo no baseline (-0,004), **não há overfitting para combater**. Weight decay é remédio para uma doença que este modelo não tem.

4. Ele ajudaria com **poucos dados** — releia o E5 do Capítulo 3: com 155 nomes, treino 0,80 contra validação 6,51. Lá o gap é enorme, e a regularização teria o que fazer.

E8 — Outros agendamentos

Agendamento	Validação
constante	1,9449
cosseno	1,9219
linear	1,9206
degraus	1,9404

Diferença entre o melhor e o pior: **0,0243**.

1 e 2. Decair ganha de não decair — mas os agendamentos não são equivalentes entre si. O linear e o cosseno ficam praticamente empatados e claramente à frente; os **degraus** ficam quase tão ruins quanto o constante.

Faz sentido: com apenas 2.500 passos, o primeiro degrau (em 50%) demora demais a chegar. Agendamentos por degrau foram feitos para treinos longos, em que cada patamar tem tempo de render.

E o cosseno ser o padrão da área é mais **convenção** do que superioridade medida — pelo menos nesta escala, o linear empata ou ganha.

3. O problema de depender de `max_steps`: o cosseno precisa saber o total de passos **de antemão**. Se você quiser continuar um treino já terminado, a curva já chegou ao fim e não há como esticá-la sem estragar o formato — a learning rate ficaria presa no mínimo.

É um incômodo real, e existem alternativas: o **WSD** (*warmup-stable-decay*) mantém a learning rate constante por tempo indeterminado e só decai no fim, permitindo decidir depois quando parar.

Gabarito — Capítulo 08

Respostas discursivas. Os **números** vêm de `gabarito.py`.

Para exercitar a GPU, rode com o Python que tem o `torch-directml`:
`C:\dml312\Scripts\python.exe solucoes/gabarito.py`

(!) **Seus números não vão bater com os daqui, e está tudo bem.** Veja a nota sobre variabilidade no E2.

E1 — Latência vs vazão

1. Por que a GPU perde numa matmul de 128×128? Porque há um **custo fixo** por operação — lançar o kernel, coordenar o dispositivo, sincronizar — que não depende do tamanho do trabalho. Numa matmul minúscula, esse custo fixo é maior que o cálculo em si. A CPU faz a conta e devolve; a GPU passa a maior parte do tempo se organizando.

2. Por que a CPU fica em ~340 GFLOP/s e a GPU vai de 167 a 5.500? A CPU **satura cedo**: ela tem poucos núcleos e já os usa quase todos mesmo em matrizes pequenas. A GPU tem **milhares** de núcleos, e só os ocupa quando há trabalho para todos. Em matrizes pequenas, a maioria fica ociosa.

3. Uma tarefa com muitos `if` e desvios roda melhor em quê? Em **CPU**. Núcleos de GPU executam em grupos que compartilham a mesma instrução; quando o código diverge (uns entram no `if`, outros não), o hardware precisa executar os dois caminhos em sequência e descartar o que não vale — perdendo boa parte do paralelismo. A CPU tem previsão de desvio e execução fora de ordem justamente para esse tipo de código.

E2 — Encontre o seu ponto de virada

Uma execução (AMD Radeon RX 7600 via DirectML):

Tamanho	CPU (ms)	GPU (ms)	Speedup	GFLOP/s GPU
128	0,05	0,03	1,52x	136
256	0,18	0,04	4,51x	857
512	1,36	0,10	13,89x	2.745
1024	7,47	0,40	18,55x	5.332
2048	62,94	3,72	16,92x	4.619

1 e 2. A GPU ganha a partir de algumas centenas, e o speedup cresce até estabilizar em torno de 15-18x.

(!) Sobre a variabilidade — e um erro que eu cometi

Estes números não batem com os da apostila (que mediu 0,75x em 128 e 15,26x em 1024). E não porque um dos dois esteja errado.

Ao preparar este gabarito eu rodei a mesma medição três vezes e obtive **1,08x, 1,52x e 2,09x** para 128×128. A variação vinha de **outros processos disputando a CPU** — o gabarito de outro capítulo estava treinando ao fundo enquanto eu media.

Medição de tempo é sensível a: - carga da máquina (o meu erro) - estado térmico e frequência do processador - até se você multiplica `a @ b` ou `a @ a` — a segunda tem melhor localidade de cache e deixa a CPU mais rápida

O que é estável, e o que você deve verificar: - a **forma** da curva: speedup baixo nos tamanhos pequenos, crescendo até estabilizar - a **ordem de grandeza** do ganho máximo (uma a duas dezenas de vezes) - o fato de que a **CPU satura** e a **GPU escala**

Se quiser números comparáveis, feche tudo e rode com a máquina ociosa. É irônico e apropriado que o capítulo sobre medir corretamente tenha me pegado justamente aqui.

3. O pico de GFLOP/s medido fica bem abaixo da especificação da placa. Isso é normal: a especificação supõe uso perfeito das unidades, precisão reduzida e nenhum gargalo de memória. Os capítulos 9 e 10 atacam exatamente essa diferença.

E3 — A armadilha do `.item()`

ms/ passo

será
ler
a
loss

lent
1,67 (+33%)

`.item()`

a
cada
passo

1 e 2. Um terço a mais de tempo, e a causa são **duas coisas** que o `.item()` faz:

- **transfere** um valor da GPU para a CPU
- para poder fazê-lo, **espera** a GPU terminar tudo o que estava na fila

A segunda é a cara. Ela destrói o paralelismo entre CPU e GPU: normalmente a CPU vai enfileirando o próximo passo enquanto a GPU trabalha no atual; com o `.item()`, ela para e espera.

3. Como registrar sem pagar o preço: - registre a cada N passos (100, por exemplo) — você precisa da **curva**, não de todos os pontos - ou acumule os valores num tensor **na GPU** e leia só no fim

E4 — Tamanho do batch

Batch	ms/ passo	ms por exemplo
32	0,149	0,00466
128	0,174	0,00136
512	0,183	0,00036
2048	0,477	0,00023

1 e 2. O tempo por passo cresce **muito menos** que proporcionalmente: batch 64x maior custa só 3x mais tempo. Por isso o tempo **por exemplo** cai 20x. É a mesma lição da Seção 4 da apostila.

3. O limite é a memória da placa — você verá um erro de alocação.

4. Cuidado com a conclusão, e este é o ponto do exercício. Batch maior processa mais exemplos por segundo, mas **cada passo continua sendo um passo de gradiente**. Se você dobrar o batch sem ajustar a learning rate, o treino faz metade do progresso por exemplo visto. Releia o E5 do Capítulo 7 e a Seção 6 do Capítulo 10.

E5 — Meça errado de propósito

Medição	Tempo	"Speedup"
GPU sem drenar	0,054 ms	230,6x
GPU com drenar	0,460 ms	26,9x

A medição errada exagera em ~9x (e numa das execuções, em 18x).

1. Sem drenar, a GPU parece absurdamente rápida — porque estamos medindo o tempo de **enfileirar** a operação, não o de executá-la. O Python devolve o controle imediatamente.

2. Por que a CPU não é afetada? Porque nela a execução é **síncrona**: quando a linha termina, a conta terminou. Não existe fila.

3. Como detectar que a medição está contaminada: - speedups que **não crescem de forma monotônica** com o tamanho - valores fisicamente implausíveis (centenas de vezes) - tempos que não

mudam quando o trabalho aumenta

E6 — Operações que caem de volta na CPU

Solução em `e6_fallback.py`:

Otimizador	GPU (ms/passo)	Speedup	Fallback
AdamW	75,2	2,65x	<code>aten::lerp.Scalar_out</code>
SGD + momentum	32,7	5,94x	nenhum
SGD puro	30,9	6,23x	nenhum

Trocar o otimizador **mais que dobra** o ganho. Aquela única operação custava ~42 ms por passo — mais da metade do tempo total. E não aparece como erro: só como lentidão.

E7 — Quando vale a pena mudar para GPU?

Juntando os números da Seção 7 da apostila:

Modelo	Parâmetros	Speedup
pequeno (o do curso)	153 mil	0,30x — a GPU perde
médio	3,2 M	2,65x
grande	18,9 M	6,82x

1. O ponto de equilíbrio fica entre 153 mil e 3,2 milhões de parâmetros — mais perto do limite inferior, já que 3,2 M já dá 2,65x.

2. O custo não é só de execução. Instalar um ambiente separado, descobrir operações não suportadas (o `lerp` do E6) e depurar travamentos consomem **horas**. Para um treino de 6 minutos, não compensa; para um de 6 horas, compensa muito. A conta é de tempo total, não de tempo de GPU.

3. Para o modelo pequeno deste curso, a recomendação é CPU — e essa é a resposta contra-intuitiva do capítulo. "Usar GPU" não é automaticamente certo: é uma decisão que depende da escala e que **se mede**.

Gabarito — Capítulo 09

Respostas discursivas. Os **números** vêm de **gabarito.py**, que roda na CPU em segundos. O **E6** depende de GPU — a resposta está abaixo, com os números que a apostila mediu.

E1 — Contas com bits

1. Qual formato representa **100000**? E **1e-9**?

- **100000** -> só o **bf16**. O fp16 estoura em 65.504.
- **1e-9** -> só o **bf16**. O fp16 zera abaixo de $\sim 6e-8$ (contando com subnormais).

Os dois casos têm a mesma causa: os 8 bits de expoente do bf16 contra os 5 do fp16.

2. Quantos valores distintos entre 1 e 2?

Com **m** bits de mantissa, exatamente **2^m** valores. Então: - bf16 (7 bits) -> **128** valores entre 1 e 2 - fp16 (10 bits) -> **1.024** valores - fp32 (23 bits) -> $\sim 8,4$ milhões

3. Por que dobrar os bits de mantissa não dobra a precisão? Porque a relação é **exponencial**, não linear: cada bit a mais **dobra** o número de valores representáveis. Ir de 7 para 14 bits multiplica a resolução por **$2^7 = 128$** , não por 2.

E2 — Achando os limites na mão

```
fp16: estoura em  $2^{15} = 3,277e+04$  (finfo.max = 6,550e+04)
      zera em  $2^{-25} = 2,980e-08$  (finfo.tiny = 6,104e-05)
bf16: estoura em  $2^{127} = 1,701e+38$  (finfo.max = 3,390e+38)
      zera em  $2^{-134} = 4,592e-41$  (finfo.tiny = 1,175e-38)
```

1. O expoente máximo bate com o **finfo.max** — a potência de 2 exata fica um pouco abaixo do máximo, que usa a mantissa cheia (**$1,111...2 \times 2^{15}$**).

2. Por que dá para ir muito abaixo do **.tiny antes de zerar?** Por causa dos números **subnormais**. Abaixo do menor valor normal, o formato abre mão de precisão para continuar representando valores cada vez menores — usa a mantissa como se o expoente estivesse travado no mínimo. É uma "rampa de saída" gradual em vez de um corte seco. Repare na distância: o fp16 tem **.tiny = 6,1e-05** mas só zera em **3,0e-08**, três ordens de grandeza abaixo.

3. O bf16 estoura e zera em expoentes muito maiores porque tem 8 bits de expoente — o mesmo alcance do fp32.

E3 — O epsilon na prática

x	$1+x$ em fp32	$1+x$ em fp16	$1+x$ em bf16
1e-1	1,100000	1,099609	1,101562
1e-2	1,010000	1,009766	1,007812
1e-3	1,001000	1,000977	1,000000
1e-4	1,000100	1,000000	1,000000

(Os valores em negrito são somas que **não mudaram nada** — o x foi arredondado para fora.)

1. Em fp16 a soma deixa de ter efeito por volta de $x = 1e-4$ (epsilon = $9,77e-04$).
2. Em bf16 o limite é **maior** (epsilon = $7,81e-03$): ele já falha em $1e-3$. Tem 3 bits menos de mantissa, então distingue menos casas decimais.
3. **É exatamente por isso que os pesos mestres ficam em fp32.** Uma atualização típica de treino tem ordem de $1e-4$ a $1e-6$. Somada a um peso de ordem 1 em 16 bits, ela é arredondada para fora e o peso **não muda** — como a apostila mede (o peso continua em $1,000000$ depois de 100 atualizações).

E4 — A janela do loss scaling

Pesos	Escala	% zerados	infs	Situação
0,02	1	50,3%	0	perde gradiente
0,02	1.024	0,1%	0	ok
0,02	2^{20}	0,0%	2	overflow
0,50	1	0,1%	0	ok
0,50	1.024	0,0%	0	ok
0,50	2^{20}	0,0%	1	overflow

1 e 2. Leia a tabela com cuidado — o efeito não é o que parece.

Com pesos maiores (0,50), o underflow praticamente **desaparece**: com escala 1, os zerados caem de 50,3% para 0,1%. O limite **inferior** da janela desce muito.

Mas o limite **superior** quase não se mexe: os dois casos começam a estourar por volta de 2^{20} . Então a janela **não se desloca — ela alarga**.

E faz sentido: o *underflow* depende da magnitude dos gradientes (que muda com os pesos), enquanto o *overflow* depende do teto do formato (65.504), que é **fixo**. Só o piso se move.

3. O scaler dinâmico continua se justificando, mas por um motivo mais preciso: a magnitude dos gradientes **cai** conforme o modelo converge. Uma escala fixa escolhida no início pode ficar pequena demais depois, reintroduzindo underflow justamente na fase de ajuste fino.

E5 — Melhorando o GradScaler

Fator	Escala final	Passos descartados
1,01	70.966	0 / 40
2,00	524.288	4 / 40
16,00	65.536	6 / 40

1. Fator **grande** (16) reage rápido mas passa do ponto: sobe demais, estoura, desce demais — e descarta 50% mais passos. Fator **pequeno** (1,01) é estável (zero descartes) mas demora a se ajustar quando a magnitude dos gradientes muda.

2. Por que um piso é útil? Ele evita que uma sequência de overflows derrube a escala a um valor tão baixo que **reintroduza underflow**. Sem piso, o scaler pode se auto-sabotar ao reagir de forma exagerada a um pico isolado.

3. Por que `growth_interval = 2000`? Porque errar para cima custa **um passo descartado**, mas errar com **frequência** faz o treino descartar passos o tempo todo. É melhor subir raramente e ficar estável do que perseguir o ótimo e viver descartando.

E6 — Seu hardware acelera 16 bits? (precisa de GPU)

Medido na máquina desta apostila (AMD Radeon RX 7600 via DirectML), matmul 1024×1024:

fp32	fp16	Ganho
CPU 35 ms	1.951 ms	~0,003x
Radeon 10 ms	0,41 ms	0,98x

1. Nesta máquina, **nenhum ganho**.

2. Isso significa que **precisão reduzida é inútil? Não** — e o exercício existe para essa distinção. São dois benefícios independentes:

- **memória**: garantido pela aritmética (metade dos bytes), vale em qualquer hardware
- **velocidade**: depende de o hardware e o backend terem unidades de 16 bits

Aqui o segundo não aparece. Numa NVIDIA com tensor cores, aparece — tipicamente 2x a 8x.

3. Se você tem NVIDIA, compare o ganho medido com a especificação da placa. Ele raramente chega ao pico teórico, porque nem toda operação do modelo é matmul.

E7 — Treino em precisão mista

Na versão CPU do `gabarito.py`, um problema de brinquedo treinado 400 passos:

Modo	Loss final
fp32	0,0463
bf16	0,0463
fp16 (sem scaler)	0,0463

1 e 3. O autocast com **bf16** chega à mesma loss do fp32 — não estraga o treino. Esperado: o bf16 tem o alcance do fp32.

2. Aqui o **fp16 sem scaler também deu idêntico** — e isso exige honestidade sobre o que o teste não mostra.

Não significa que loss scaling seja dispensável. Significa que **este problema não provoca underflow**. Compare com o `loss_scaling.py` da apostila: lá os pesos são deliberadamente pequenos (`p.mul_(0.02)`), os gradientes ficam na casa de `1e-8`, e **50,3% deles zeram**. Aqui os pesos são os da inicialização padrão, os gradientes são saudáveis, e nada zera.

A lição metodológica: um teste que passa não prova que a proteção é desnecessária — prova que o teste não exercita o caso que ela protege. É a mesma armadilha do E2 do Capítulo 4, com a máscara causal.

Modelos profundos de verdade têm gradientes muito menores nas camadas iniciais, e é lá que o fp16 sem scaler quebra.

4. **Por que a economia de memória não chega aos 50%?** Porque a cópia **mestra** dos pesos e o estado do otimizador (`m` e `v` do AdamW) continuam em fp32. Só as ativações e as matmuls usam 16 bits.

Gabarito — Capítulo 10

Respostas discursivas. Os **números** vêm de `gabarito.py`, medidos numa máquina de 12 núcleos.

```
python solucoes/gabarito.py
```

O E3 roda em **subprocesso com timeout**, porque o objetivo literal daquele exercício é fazer o programa travar — e um gabarito que trava para sempre não serve para ninguém.

E1 — Entendendo o coletivo

1. Quatro processos, o rank **i** tem o valor **i**.

Operação	Resultado em cada rank
<code>all_reduce</code> com SUM	$0+1+2+3 = 6$
<code>all_reduce</code> com MAX	3

O ponto do "all": **todos** terminam com o mesmo valor. Não é o rank 0 que fica com a soma — é todo mundo.

2. As quatro operações.

Quem fica com o resultado
<code>reduce</code> um rank (o destino)
<code>all_reduce</code> todos
<code>gather</code> um rank recebe a lista completa
<code>all_gather</code> todos recebem a lista completa

`reduce/all_reduce` **combinam** os valores (soma, máximo). `gather/all_gather` **empilham** sem combinar: cada rank fica com os N tensores separados.

3. Por que o **broadcast** não serve para sincronizar gradientes.

Porque o `broadcast` copia o valor de **um** rank para os demais — e nenhum rank tem o valor certo para transmitir. Cada um calculou o gradiente do **seu** pedaço do batch; o gradiente correto é a **média dos quatro**, que não existe em lugar nenhum antes de alguém somá-los.

Usar `broadcast` do rank 0 equivaleria a jogar fora os dados dos outros três processos — todo o trabalho deles seria descartado. É exatamente o que a soma do `all_reduce` evita.

E2 — Escalando o número de processos

Tempo de um `all_reduce`, melhor de 3 rodadas, **duas passadas independentes** (12 núcleos):

Processos	10 mil	1 milhão	10 milhões
2	0,54 / 0,56 ms	21,0 / 15,3 ms	264,9 / 280,2 ms
4	2,35 / 1,83 ms	9,8 / 8,7 ms	93,6 / 105,8 ms
8	5,81 / 5,70 ms	13,3 / 12,7 ms	174,2 / 177,7 ms

Reprodutibilidade entre passadas: **1,02x a 1,38x**. O efeito discutido abaixo é de **2,7x** — bem acima do ruído.

1. Não há uma resposta só — ela depende do tamanho do tensor, e é essa a descoberta.

- **10 mil elementos:** o tempo **crece ~10x** de 2 para 8 processos.
- **10 milhões:** o tempo **cai quase 3x** de 2 para 4 processos.

Sim, *cai*. Com mais processos, o all-reduce do tensor grande fica **mais rápido**.

Eu tinha escrito que o tempo sempre cresce com N, apoiado na propriedade do anel (cada processo troca ~2× o tamanho do tensor, independentemente de N). Essa propriedade é verdadeira — e não é a que domina.

A explicação é a mesma dicotomia do [Capítulo 8](#), **latência vs vazão**:

O que domina	Efeito de mais processos
Latência: dos N-1 saltos do anel pequeno	mais saltos -> pior
Vazão: transferir e somar 40 MB grande	trabalho dividido -> melhor

No caso grande, o anel divide o trabalho em pedaços de `tamanho/N`. Com N=2 cada processo lida com 20 MB por etapa; com N=4, com 10 MB. Pedaços menores cabem melhor na cache, e a **soma** — que é trabalho real, não só transferência — se espalha por mais núcleos.

2. No esquema ingênuo (todos -> rank 0 -> todos), o rank 0 receberia $(N-1) \times \text{tamanho}$: o tempo cresceria **linearmente** com N, e ele viraria um gargalo cada vez pior. E o ganho observado em N=4 seria **impossível** — o rank 0 é um só, e não paraleliza.

3. Onde os processos passam a competir por CPU. Nesta máquina de 12 núcleos, N=8 perde para N=4 nas colunas grandes: o tempo piora por **contenção**, não por comunicação. Numa máquina só, "mais processos" tem um teto físico, e ele fica perto do número de núcleos.

(!) **Cuidado ao generalizar.** Tudo isto foi medido em *uma* máquina, com gloo em CPU. Num cluster de verdade, com GPUs e rede dedicada, o equilíbrio entre latência e vazão é outro — mas a **pergunta a fazer** é a mesma.

E3 — Provoque um deadlock

Este exercício me pegou. Vale ler a previsão antes do resultado.

A previsão (errada)

A explicação clássica do deadlock em `send/recv` é o **buffer de socket**: o sistema operacional aceita mensagens pequenas e devolve o controle imediatamente, sem esperar ninguém receber. Sob essa teoria, tensores pequenos passariam por acidente e só os grandes travariam — o que faria deste um bug clássico de "passa no teste, quebra em produção".

Eu escrevi essa resposta antes de medir. O próprio enunciado original do exercício dava essa dica.

A medição

Tensor	Alternância par/ímpar	Resultado
1 elemento	sim	terminou
1 elemento	não	TRAVOU
100 elementos	sim	terminou
100 elementos	não	TRAVOU
5 milhões	sim	terminou
5 milhões	não	TRAVOU

Trava sempre. Até com **um único elemento**. O `dist.send` do gloo só retorna quando o `recv` correspondente foi postado do outro lado — não existe tamanho pequeno o bastante para escapar.

A explicação do buffer de socket vale para **sockets crus e para MPI**. Não vale para o gloo, que fez outra escolha de projeto.

O que aprender com isso

Sobre o gloo: a escolha dele é boa. Um deadlock que acontece *sempre* é muito melhor que um que só aparece em produção com tensores grandes. O gloo trocou "às vezes funciona" por "nunca funciona", e **falhar de forma determinística é um recurso, não um defeito** — o bug aparece no primeiro teste que você rodar, na sua máquina, com dados de brinquedo.

Sobre você (e sobre mim). Eu tinha um modelo mental correto e o apliquei a uma implementação que não o segue. O modelo estava certo *em geral* e errado *naquele caso* — e nenhuma quantidade de raciocínio teria revelado isso. Só medir revela.

É o mesmo padrão do [E2 do Capítulo 4](#) e do [E3 do Capítulo 5](#).

E note o modo de falha: não há exceção, não há mensagem. O programa simplesmente para — exatamente como aconteceu no desenvolvimento deste capítulo, com o hostname que resolvia para um IP público (Seção 8 da apostila).

E4 — A loss que engana

Solução em [e4_loss_correta.py](#).

1. Por que os ranks imprimem losses diferentes se os pesos são idênticos? Porque cada rank calcula a loss no **seu** micro-batch, e os micro-batches são diferentes. Os *pesos* são idênticos (o DDP garante isso sincronizando os gradientes); os *dados* não.

2. Com `all_reduce` da loss dividido por `world_size`, os valores passam a bater entre os ranks — porque agora todos estão reportando a mesma quantidade: a loss média sobre o batch global.

3. O que aconteceria com a curva se você registrasse só o rank 0. Ela ficaria **mais ruidosa** do que o treino realmente é. A loss do rank 0 é uma amostra de batch 64; a loss global é uma amostra de batch 256. Quatro vezes mais dados, metade do ruído.

O perigo prático: você olha a curva serrilhada, conclui que o treino está instável, e sai mexendo na learning rate para consertar um problema que **existe só na medição**.

E5 — Batch efetivo e learning rate

300 passos, batch **local** fixo em 64:

Processos	Batch efetivo	lr	Loss global
1	64	<code>1e-3</code>	2,0095
2	128	<code>1e-3</code>	1,9417
4	256	<code>1e-3</code>	1,8494
2	128	<code>1,4e-3</code> ($\times\sqrt{2}$)	1,9155
4	256	<code>2,0e-3</code> ($\times\sqrt{4}$)	1,7846

1. Com o batch local fixo, mais processos = batch efetivo maior. Como o número de **passos** é o mesmo, o modelo vê mais dados — mas dá o mesmo número de **atualizações**. A loss melhora, porque cada gradiente é menos ruidoso.

2. Sim, ajustar a lr melhora ainda mais. Escalar por `world_size` recuperou parte da diferença nos dois casos (1,9417 -> 1,9155 e 1,8494 -> 1,7846). Gradiente menos ruidoso tolera — e pede — passo maior.

3. Por que "mais GPUs" pode deixar o treino mais lento em número de passos. Porque você processa mais exemplos por passo, mas continua andando com o **mesmo tamanho de passo**. Mais dados por atualização, mesmo número de atualizações: é preciso mais passos para chegar ao mesmo lugar. O ganho está em tempo de parede por passo, não em progresso por passo — e só se materializa se você ajustar a learning rate junto.

É a mesma armadilha do [E4 do Capítulo 8](#) (batch maior é mais eficiente por exemplo, mas não é grátis do ponto de vista da otimização).

E6 — Meça o ZeRO

Estado do otimizador, por processo:

Otimizador	Processos	Normal	ZeRO-1	Redução
AdamW	2	8,54 MB	4,33 MB	2,0x
AdamW	4	8,54 MB	2,23 MB	3,8x
SGD sem momentum	2	0,00 MB	0,00 MB	—
SGD sem momentum	4	0,00 MB	0,00 MB	—

1. Sim, o estado por processo cai para $\sim 1/N$ — como a apostila mede com 4 processos.

2. E aqui está o ponto do exercício: com SGD sem momentum, o ZeRO-1 não economiza nada. Zero contra zero.

O motivo é direto: o SGD puro **não guarda estado nenhum** — ele usa o gradiente e descarta. Se o otimizador não guarda nada, não há o que fatiar. O ZeRO-1 economiza exatamente aquilo que o otimizador guarda.

(Os estágios 2 e 3, que fatiam **gradientes** e **pesos**, continuariam ajudando — eles não dependem do otimizador.)

3. Com bf16 nos pesos (Capítulo 9), a conta muda de forma interessante. Pesos e gradientes passam de 4 para 2 bytes por parâmetro, mas o estado do AdamW costuma ficar em fp32 por precisão. Com 8 processos:

Pesos	Gradientes	Estado	Total/GPU
2 GB	2 GB	8 GB	12 GB

ZeRO	2 GB	1 GB	5 GB
------	------	------	------

O peso **relativo** do estado do otimizador **aumenta** quando os pesos encolhem — ou seja, precisão reduzida torna o ZeRO ainda mais valioso. As duas técnicas se reforçam.

E7 — Quando distribuir *não* compensa

Juntando os números medidos:

- tempo de cálculo de um passo (modelo pequeno, CPU, Capítulo 8): **~12,8 ms**
- all-reduce de ~150 mil parâmetros (0,6 MB): a coluna "10 mil" do E2, escalada — fica na casa de **alguns milissegundos**

1 e 2. Para o modelo pequeno deste curso, comunicação e cálculo ficam na **mesma ordem de grandeza**. Distribuir em 4 processos dividiria o cálculo por 4 mas acrescentaria comunicação a cada passo — ganho líquido pequeno ou negativo.

3. A regra geral:

Distribuir compensa quando **tempo de cálculo por passo >> tempo de comunicação por passo**

O cálculo cresce com o **tamanho do modelo** e com o **batch**. A comunicação cresce só com o tamanho do **modelo** — o all-reduce troca gradientes, e há um gradiente por parâmetro, não por exemplo.

Logo: **aumentar o batch melhora a razão de graça**. É por isso que treino distribuído de verdade usa batches enormes — não é só por velocidade, é para que a comunicação valha a pena.

E é a mesma conclusão contra-intuitiva do [E7 do Capítulo 8](#): para o modelo deste curso, a resposta certa é **não distribuir**.

Gabarito — Capítulo 11

Respostas discursivas. Os **números** vêm de `gabarito.py`.

```
bash python solucoes/gabarito.py
```

Sobre o orçamento de treino — leia antes dos números

O modelo da apostila leva **~18 minutos** por treino (3.000 passos). Este gabarito roda vários experimentos, então usa **400 passos**. A arquitetura é **idêntica** à da apostila (`n_embd=192`, 6 cabeças, 4 blocos, 2,2 M parâmetros) — só o número de passos muda.

Essa distinção importa. Reduzir passos é defensável para perguntas **comparativas**; trocar a arquitetura não seria, porque mudaria o objeto em estudo.

Mas "defensável" não é "seguro". A aposta implícita é que a **ordem** entre as configurações não depende do orçamento, mesmo que os valores absolutos dependam — e essa aposta **já perdeu duas vezes neste curso**:

Onde	Com orçamento curto	Com orçamento cheio
Cap. 3, E4	melhor <code>lr</code> = 1,0	melhor <code>lr</code> = 0,1
Cap. 7, E5	<code>3e-3</code> ganha	<code>1e-3</code> ganha

Nos dois casos a pergunta era sobre **dinâmica de otimização** — e o orçamento é justamente a variável que a dinâmica consome. As perguntas *deste* capítulo são **estruturais** (quanto contexto, quantas posições, quantos dados), e deveriam ser estáveis.

"Deveriam" não é medição. Rode `_checagem_orcamento.py`, que repete o E4 com o triplo dos passos e compara os **rankings**. É o mesmo teste que você deve aplicar a qualquer experimento seu que rode com orçamento reduzido.

E1 — Por que essa ordem?

1. Por que o tokenizador é treinado só no texto de treino?

Porque o BPE **aprende com os dados**: ele decide quais pares de bytes fundir contando frequências. Se ele vir o texto de validação, essas contagens incluem material que o modelo deveria não conhecer.

O que vazaria, concretamente: se `Memorial de Aires` entrasse no treino do tokenizador, ele criaria tokens para nomes e expressões daquele livro. Na hora de avaliar, o texto de validação ficaria **artificialmente mais barato de codificar** — menos tokens, cada um mais previsível — e a loss cairia sem que o modelo tivesse ficado melhor.

É um vazamento sutil porque o tokenizador não é "o modelo". Mas ele faz parte do **pipeline**, e tudo que toca o pipeline precisa respeitar a divisão.

2. Por que dividir por obra e não sorteando parágrafos?

Porque parágrafos da mesma obra **compartilham informação**. Um exemplo concreto: se um parágrafo de *Dom Casmurro* que menciona "José Dias" cair no treino e o parágrafo seguinte cair na validação, o modelo já viu o nome, o assento narrativo, o vocabulário da cena e até a construção da frase anterior. Ele acerta a validação por ter visto quase a mesma coisa — não por ter aprendido português.

Dividir por obra garante que a validação seja um texto que o modelo **nunca viu em nenhuma forma**. É a diferença entre "consegue continuar este livro" e "consegue escrever como Machado".

O **E2** mede exatamente esse efeito.

3. Por que **uint16** e não **int64**?

Porque o vocabulário tem 1.024 entradas e cabe folgado em 16 bits (que aguentam 65.536). O **int64** guardaria os mesmos números em **8 bytes** em vez de 2:

Bytes/token	621 mil tokens
int64	4,97 MB
uint16	1,24 MB

Quatro vezes menos espaço, e o ganho não é só de disco: é de **banda de memória**. Como o treino lê os dados por **memmap** a cada batch, um arquivo 4x menor significa 4x menos tráfego entre disco, cache e CPU. Em corpora de verdade — centenas de gigabytes — essa escolha decide se o treino é limitado pelo cálculo ou pela leitura dos dados.

(!) O limite é real: com vocabulário acima de 65.536, o **uint16 estoura silenciosamente** (os índices dão a volta). Tokenizadores modernos usam 100 mil a 200 mil tokens e precisam de **uint32**.

E2 — A divisão errada

Solução em **e2_vazamento.py**.

E3 — O tokenizador é do domínio

1. Os 20 tokens mais longos aprendidos:

```
'José Dias ' | 'ima Justin' | 'que não ' | 'minha mã' | 'Capitú, '
'os olhos ' | 'da minha ' | 'inha mã' | 'Capitú ' | 'José Di'
'a minha ' | 'seminari' | 'ação, ' | 'almente ' | 'não me '
'que era ' | 'Capitú' | 'ação ' | 'é que ' | 'primeir'
```

6 dos 20 contêm nome de personagem (**José Dias**, **Capitú**, **Justin[a]**). Vários outros são marcas do texto específico: **'minha mã[e]'**, **'seminari[o]'**, **'os olhos '** — o narrador de *Dom Casmurro* falando da mãe, do seminário e dos olhos de ressaca.

Ninguém programou isso. O BPE só contou pares de bytes.

2. Isso é bom ou ruim? Depende de onde você vai usá-lo.

É **ótimo para este corpus e péssimo fora dele**. Um tokenizador que gasta um único token em **'José Dias '** comprime Machado maravilhosamente — e não tem nenhuma fusão útil para um artigo de medicina, onde cada **'hemoglobina'** seria fatiado em pedaços.

3. O BPE treinado em nomes, aplicado a Machado.

Mesmo trecho de prosa (109 bytes), dois BPEs de vocabulário 1.024:

BPE treinado em	Tokens	Chars/token
Machado	43	2,53
nomes (Cap. 6)	81	1,35

1,88x mais tokens para dizer a mesma coisa. O BPE de nomes aprendeu terminações como **'ilson'** e **'erson'**, que quase não aparecem em texto corrido — e não aprendeu nada sobre **que**, **de** ou **cão**.

O custo é direto: 1,88x mais tokens significa 1,88x mais posições de contexto gastas para o mesmo texto, e um treino proporcionalmente mais caro para o mesmo conteúdo.

Nota de método. A primeira versão deste gabarito comparava "81 tokens no trecho" (de um tokenizador) com "2,59 chars/token no corpus inteiro" (do outro). São **grandezas diferentes**, e a comparação parecia razoável sem ser. O número acima mede os dois no mesmo trecho.

E4 — Tamanho do contexto

400 passos, modelo da apostila:

block_size	Treino	Validação	ms/ passo
32	4,6154	4,8286	103,2
128	4,4455	4,6957	426,4
256	4,4013	4,6464	878,3

1. Mais contexto melhora a loss — e isso é o **contrário** do que acontecia no [Capítulo 4](#), onde **block_size=16** era pior que 8.

A diferença é o dado, não o modelo. Nomes têm ~7 letras: não há nada a lembrar de longe, e contexto extra só adiciona ruído. Prosa tem dependências longas de verdade.

2. Dobrar o contexto quadruplica o tempo? Não — e a resposta surpreende.

Mudança	Contexto	Tempo	Se fosse linear	Se fosse quadrático
32 -> 128	4x	4,13x	4x	16x
128 -> 256	2x	2,06x	2x	4x

O crescimento é **linear**, apesar de a atenção ser $O(T^2)$.

O motivo: a atenção é **só uma parte** do custo. As camadas lineares — **qkv**, projeção, e o feedforward que tem 4x a largura do modelo — crescem **linearmente** com T, e neste tamanho elas dominam. O termo quadrático existe, mas só passa a mandar em contextos bem maiores.

É por isso que "atenção é quadrática" é verdade e é enganoso ao mesmo tempo: verdade sobre a fórmula, enganoso sobre onde o seu tempo está indo. Meça.

3. **Por que o contexto ajuda mais para prosa.** Porque a dependência linguística é longa: concordância, referência a personagens, estrutura de período, o assunto do parágrafo. Tudo isso liga tokens distantes. Um nome de 7 letras não tem nada disso.

E5 — Prever em todas as posições

Modo	Treino	Validação	Previsões/batch	ms/passo
todas as posições	4,4455	4,6957	4.096	365,1
só a última	6,0319	6,0564	32	360,4

1. Com **batch_size=32** e **block_size=128**: **4.096** previsões por batch contra **32**. São **128x mais sinal de treino**.

2. **A diferença é enorme**: 4,70 contra 6,06 de loss de validação, com o mesmo orçamento de passos. A versão que prevê só na última posição recebe 128x menos informação por passo.

3. **E o custo é praticamente o mesmo** — 365 ms contra 360 ms.

Esse é o ponto do exercício. As duas versões processam as 128 posições de qualquer jeito; a única diferença é **quantas saídas da camada final entram na loss**. A versão "só a última" faz o cálculo inteiro e **joga fora 99% dele**.

128x mais sinal, custo igual. É o tipo de mudança que quase não existe em engenharia — normalmente se paga por ganho.

Os capítulos 3 a 7 usaram a versão ineficiente **de propósito**, porque ela mantinha a métrica comparável entre capítulos. A partir daqui, não há motivo para isso.

E6 — Dados sintéticos e *model collapse*

Treino	Validação real
modelo A (corpus de Machado)	4,6957
modelo B (60 mil tokens gerados pelo A)	5,1453

1 e 2. O modelo B fica pior na validação real — degradação de **+0,4496**. O fenômeno tem nome na literatura: *model collapse*.

O detalhe que torna isso perigoso

Olhe a coluna de treino. A loss de treino do modelo B é **menor** que a do A (4,30 contra 4,45). Se você olhasse só a curva de treino, concluiria que o B está aprendendo melhor.

Não está. Texto gerado por um modelo é **mais previsível** que texto humano — menos vocabulário, menos construções raras, menos surpresa. Ficar bom em prever texto fácil não é ficar bom em prever Machado.

O colapso chega disfarçado de progresso. É por isso que a validação precisa vir de dados reais e intocados — o mesmo princípio do E1 e do E2 deste capítulo.

Por que acontece

A razão é informacional. O texto gerado por A é uma **amostra** da distribuição que A aprendeu — e amostrar **perde a cauda**. Padrões raros aparecem pouco ou não aparecem, e o B aprende uma versão ainda mais concentrada. Repetindo o ciclo, a diversidade colapsa.

Nenhum modelo pode aprender dos próprios dados mais do que já sabe. A geração não cria informação nova sobre o mundo.

3. Quando dados sintéticos ajudam de verdade: quando existe uma **fonte de verdade externa** para filtrar.

Domínio	O verificador
matemática	dá para conferir a resposta
código	dá para rodar os testes
jogos	dá para saber quem ganhou

Nesses casos o modelo gera muitas tentativas e um verificador seleciona as boas. A informação que entra vem do **verificador**, não do modelo — e por isso o ciclo não colapsa. É a base do que o [Capítulo 15](#) vai fazer.

E7 — Escalando o corpus

Versão do gabarito: frações do mesmo corpus

Corpus	Tokens	Treino	Validação	Gap
25%	155.283	4,3627	4,7601	0,3975
50%	310.567	4,4252	4,7330	0,3077
100%	621.134	4,4455	4,6957	0,2501

1 e 2. Mais dados melhoram a validação, com **retorno decrescente**: dobrar de 25% para 50% rendeu 0,027 de loss; dobrar de novo rendeu 0,037. Ganhos na mesma casa para um esforço que dobra.

3. A lógica do Capítulo 3 vale em direção, não em magnitude.

O **gap** entre treino e validação encolhe monotonicamente (0,40 -> 0,31 -> 0,25), confirmando que parte do problema era memorização — exatamente como lá.

Mas compare a escala:

Mudança nos dados	Efeito no gap
Cap 3 55 -> 64.000 nomes	5,7 -> ~0
Cap 11 55 mil -> 621 mil tokens	0,40 -> 0,25

Lá o modelo tinha ~10 exemplos por parâmetro; aqui já tem centenas de milhares de tokens. **Melhorias ficam progressivamente mais caras** — é a forma de toda curva de escala, e é por isso que os laboratórios discutem *ordens de grandeza* de dados, não porcentagens.

Versão literal: acrescentando obras de verdade

O enunciado pede para **acrescentar obras**. A lista completa de Machado no Gutenberg tem 12 (confirmado na API, não chutado) — o `prepare_data.py` usa 5. `e7_mais_obras.py` baixa as outras 7 e mede.

Duas decisões de método, que importam mais que o resultado:

1. **O tokenizador fica fixo.** Usamos o `tokenizador.pkl` já treinado, sem retreinar. Se retreinássemos, mudaríamos **duas coisas ao mesmo tempo** (mais dados e outro tokenizador) e não saberíamos a qual atribuir a diferença. Custo honesto: o tokenizador fica levemente mal-adaptado às obras novas — é o preço de isolar a variável, e é o preço certo a pagar.
2. **A validação não muda.** *Memorial de Aires* continua fora do treino. Sem isso as losses não seriam comparáveis entre configurações.